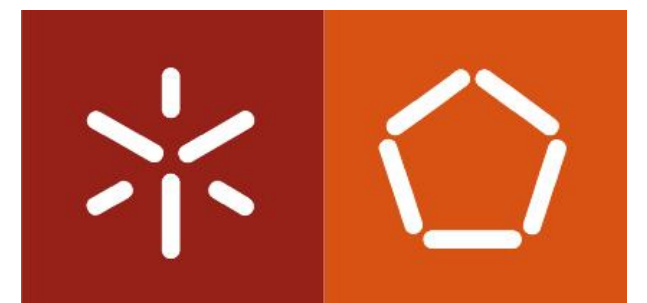


Operating Systems

(Sistemas Operativos)

Guide 5: FIFOs



Process API

Anonymous Pipes (recap)

```
1 int main() {
2     pid_t pid;
3     int fildes[2];
4     pipe(fildes);
5     if ((pid = fork()) == 0) {
6         close(fildes[0])
7         // child process
8     } else {
9         close(fildes[1])
10        // parent process
11    }
12    return 0;
13 }
```

Parent FD Array

0
1
2
3
4

Parent can read data with
`read(fildes[0], ...)`

fildes[0]

Child FD Array

0
1
2
3
4

Child can write data with
`write(fildes[1], ...)`

fildes[1]

Process API

Anonymous Pipes (recap)

How can non-related processes communicate?

```
1 int main() {  
2     pid_t pid;  
3     int fildes[2];  
4     pipe(fildes);  
5     if ((pid = fork()) == 0) {  
6         close(fildes[0])  
7         // child process  
8     } else {  
9         close(fildes[1])  
10        // parent process  
11    }  
12    return 0;  
13 }
```

Parent FD Array

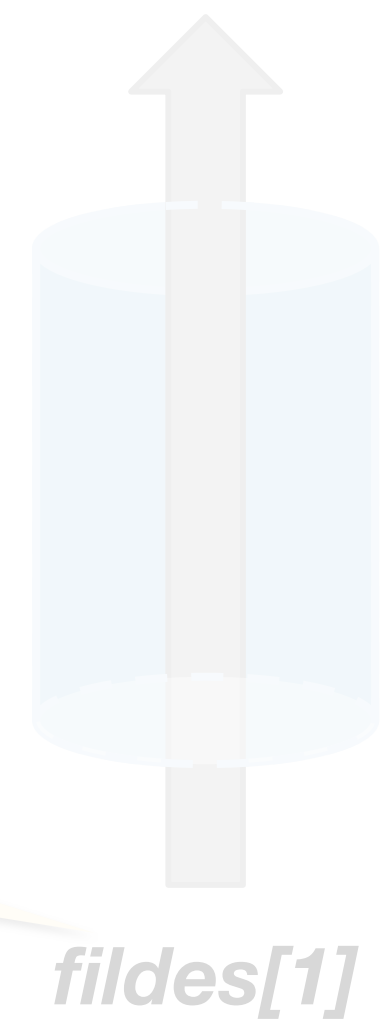
0
1
2
3
4

Parent can read data with
read(fildes[0], ...)

Child FD Array

0
1
2
3
4

Child can write data with
write(fildes[1], ...)

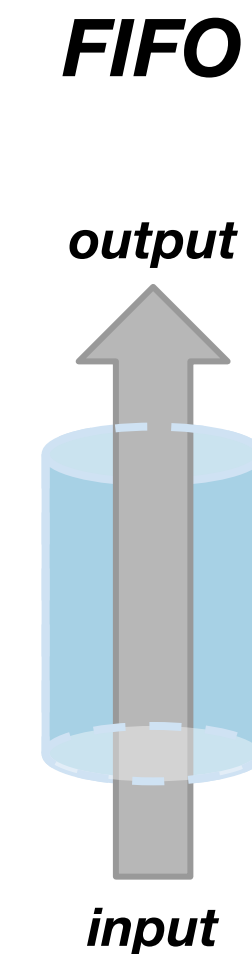


Process API

Inter-Process Communication

Named Pipes

- **Communication across non-related processes**
- Produced (written) data is **kept in a memory region** to be consumed (read).
- The kernel handles writers (producers) and readers (consumers).
- **Writers block (wait)** if there is no available space, and **readers block** if there is no data.
- Data flows in a **one-way First-in First-out (FIFO)** manner.



Process API

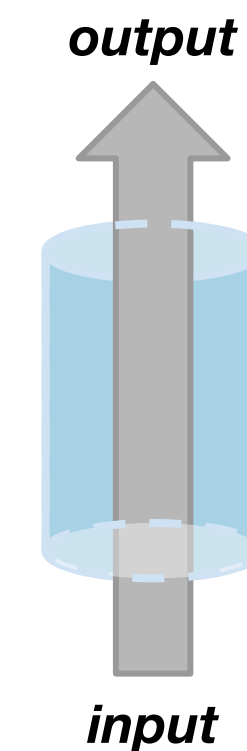
Named Pipes

```
#include <sys/types.h>
#include <sys/stat.h>
```

- *int* **mkfifo**(*const char path** *pathname*, *mode_t mode*)
 - **pathname**: absolute or relative pathname to the special FIFO file
 - **mode**: file permissions
 - 0600 - owner of the file can read/write
 - Returns: 0 on success, -1 otherwise

For more information: *\$ man 3 mkfifo*

pathname

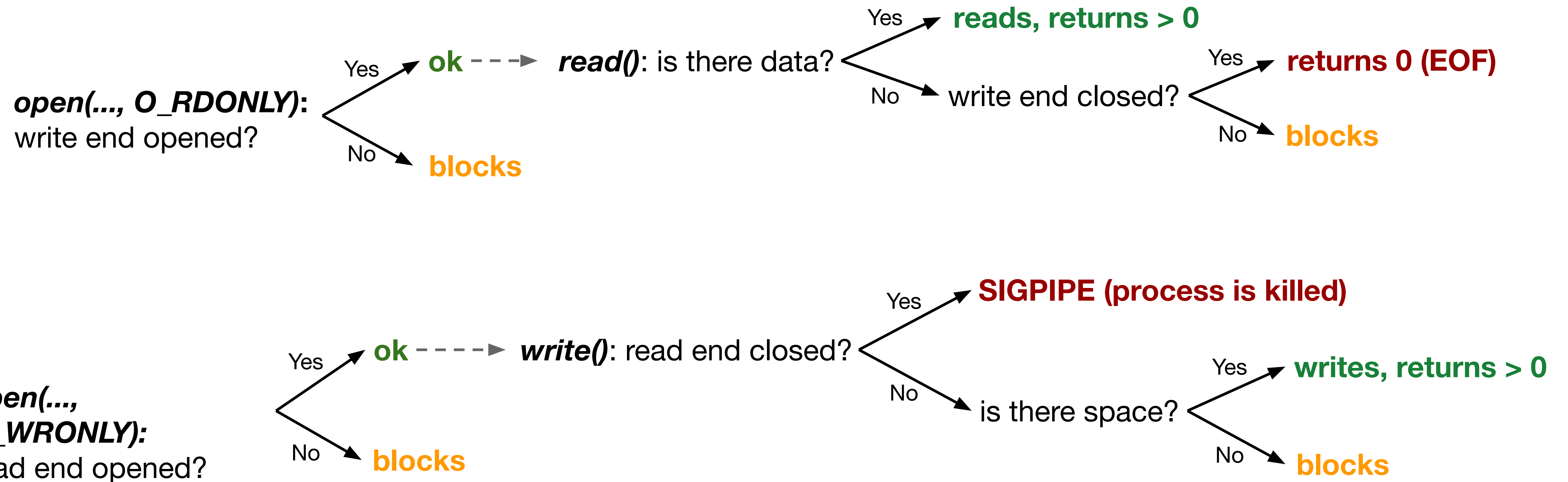


Considerations:

1. **mkfifo creates the FIFO special file**
2. **Accessed through syscalls** (*open*, *close*, *read*, *write*)
 - a. **Explicitly removed with** *unlink*
 - b. **Cannot** change file pointer (*lseek*)
3. **Unidirectional communication**
4. **Reads and writes block**

Process API

Named Pipes



Process API

Example: communicating via FIFO

PID 2034 (server)

```
→ int main() {  
2   mkfifo("fifos/myfifo", 0666);  
3   int fifo = open("fifos/myfifo", O_RDONLY);  
4   char buf[BUFSIZE];  
5   int n;  
6   while((n = read(fifo, buf, BUFSIZE)) > 0) {  
7       ...  
8   }  
9   close(fifo);  
10  unlink(fifo);  
11 }
```

FD Array

0
1
2
3

Filesystem

```
/  
|-- home/  
    |-- username/  
        |-- Project  
            |-- client  
            |-- server  
            |-- fifos/  
...
```

PID 5391 (client)

```
1 int main() {  
2   int fifo = open("fifos/myfifo", O_WRONLY);  
3   ...  
4   write(fifo, buf, BUFSIZE);  
5   close(fifo);  
6 }
```

Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {  
2     mkfifo("fifos/myfifo", 0666);  
3     int fifo = open("fifos/myfifo", O_RDONLY);  
4     char buf[BUFSIZE];  
5     int n;  
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {  
7         ...  
8     }  
9     close(fifo);  
10    unlink(fifo);  
11 }
```

FD Array

0
1
2
3

PID 5391 (client)

```
1 int main() {  
2     int fifo = open("fifos/myfifo", O_WRONLY);  
3     ...  
4     write(fifo, buf, BUFSIZE);  
5     close(fifo);  
6 }
```

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...

```

Created FIFO

Check it: `$ ls -l`
(How do you know it is a pipe?)

Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

no write end opened
blocks...

FD Array

0
1
2
3

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

FD Array

0
1
2
3

Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

write end is opened
continue

FD Array

0
1
2
3

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

read end is opened
continue

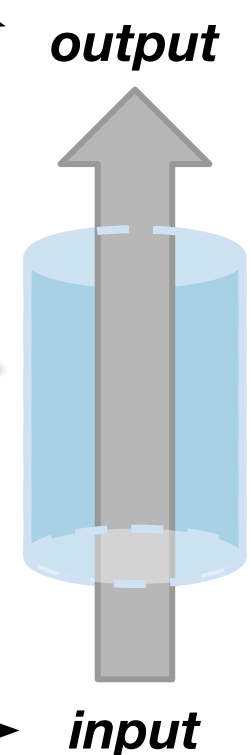
PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

FD Array

0
1
2
3

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

nothing to read
blocks...

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

FD Array

0
1
2
3

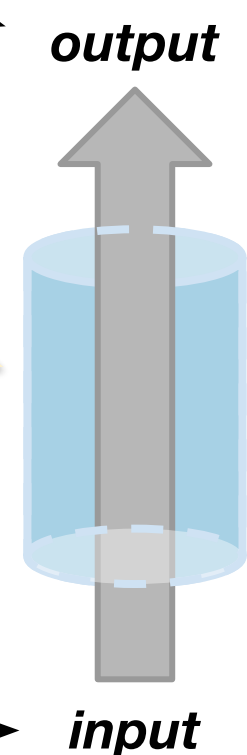
PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

FD Array

0
1
2
3

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

reads data from FIFO

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

0
1
2
3

PID 5391 (client)

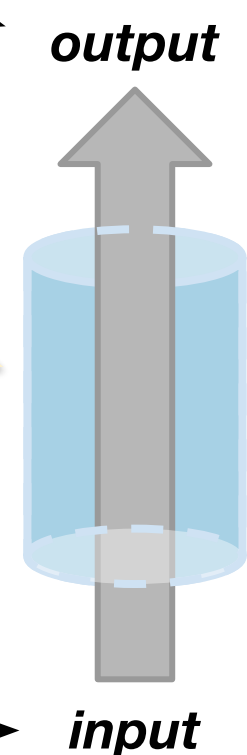
```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

writes data to FIFO

FD Array

0
1
2
3

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

blocks again

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

0
1
2
3

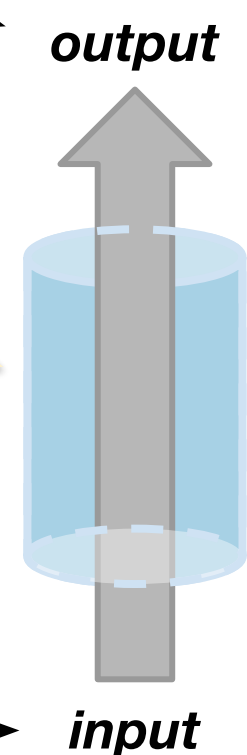
PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

FD Array

0
1
2
3

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

unblocks and read
returns 0 (EOF)

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

0
1
2
3

PID 5391 (client)

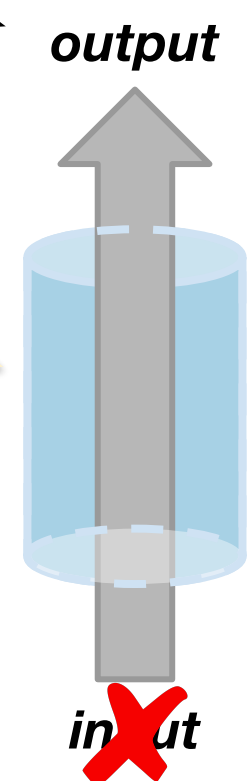
```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

closes FIFO
(without deleting it)

FD Array

0
1
2
3

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

**closes FIFO
(without deleting it)**

FD Array

0
1
2
3

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```

PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```

FIFO in main memory



Process API

Example: communicating via FIFO

PID 2034 (server)

```
1 int main() {
2     mkfifo("fifos/myfifo", 0666);
3     int fifo = open("fifos/myfifo", O_RDONLY);
4     char buf[BUFSIZE];
5     int n;
6     while((n = read(fifo, buf, BUFSIZE)) > 0) {
7         ...
8     }
9     close(fifo);
10    unlink(fifo);
11 }
```

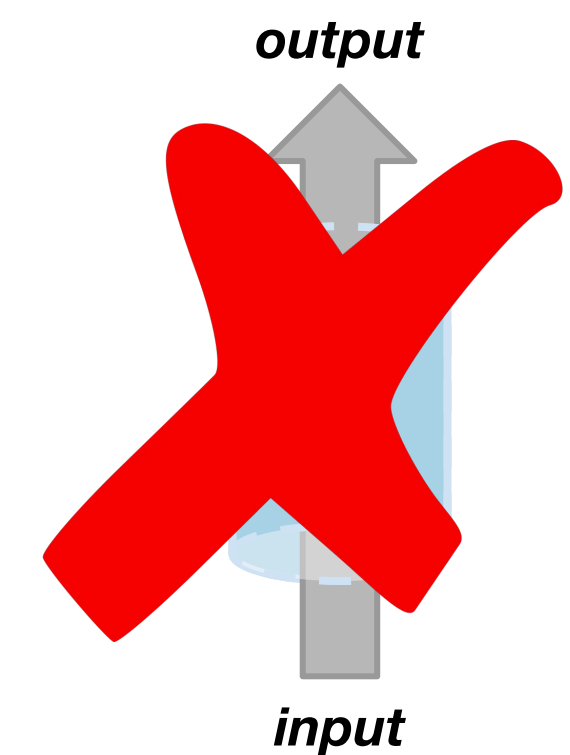
Deletes FIFO after all processes close it

FD Array

0
1
2
3

Filesystem

```
/
|-- home/
|   |-- username/
|       |-- Project
|           |-- client
|           |-- server
|           |-- fifos/
|               |-- myfifo
...
```



PID 5391 (client)

```
1 int main() {
2     int fifo = open("fifos/myfifo", O_WRONLY);
3     ...
4     write(fifo, buf, BUFSIZE);
5     close(fifo);
6 }
```